

TRABAJO FINAL DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN PLAN DE TRABAJO

Diseño e implementación de una plataforma de orquestación de agentes de lenguaje para el desarrollo de software y análisis exploratorio de una política de descomposición adaptativa

Alumno: Francisco Clarke
LU: 120547
Directora: Ana Gabriela Maguitman
Codirector: Federico Martín Schmidt

1. Objetivos

Los agentes basados en modelos de lenguaje han demostrado capacidad para interactuar con repositorios de software, modificar archivos y ejecutar herramientas de desarrollo. Trabajos como SWE-agent muestran que la interfaz provista al agente y la forma en que se estructura su interacción con el entorno influyen en su desempeño [1]. Otros enfoques, como ChatDev y MetaGPT, exploran la colaboración entre agentes especializados para cubrir distintas etapas del desarrollo de software [2, 3].

Sin embargo, una funcionalidad de software de alcance medio o grande no se reduce a una única tarea de generación de código. Es necesario interpretar el objetivo, dividirlo en unidades ejecutables, explicitar dependencias y contratos, coordinar trabajo potencialmente paralelo, verificar los cambios y recomponer los resultados sin perder trazabilidad. La ejecución paralela basada en grafos de dependencias ya ha mostrado ventajas de latencia y costo en otros dominios de orquestación de herramientas con modelos de lenguaje [4]; su aplicación al desarrollo de software agrega desafíos vinculados con el estado compartido del repositorio, la validación del código y la integración semántica de cambios.

Este trabajo propone el diseño, implementación y análisis de una plataforma denominada **Conductor**, que transforma una especificación en lenguaje natural en un proceso de desarrollo supervisable. Su arquitectura se organiza alrededor de tres mecanismos principales: descomposición recursiva, ejecución aislada e integración incremental. Cada mecanismo constituye un problema de diseño propio y su articulación permite recorrer el ciclo completo desde la intención inicial hasta la obtención de cambios integrados y verificados.

La Figura 1 resume este flujo y los mecanismos transversales de supervisión y persistencia que permiten inspeccionarlo y reanudarlo.

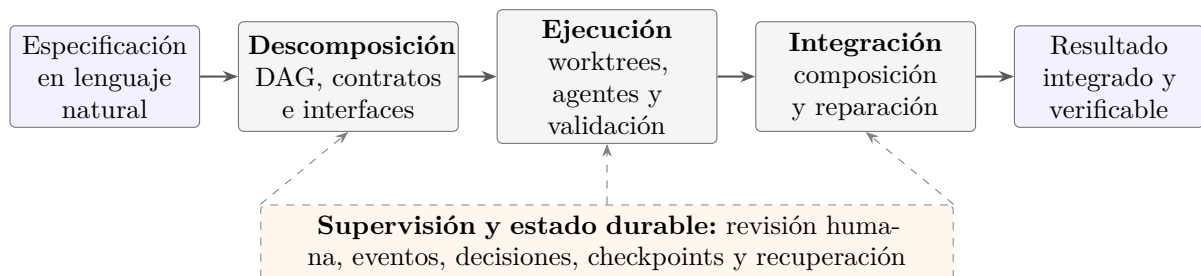


Figura 1: Arquitectura conceptual de Conductor.

Objetivo general

Diseñar, implementar y analizar Conductor, una plataforma de orquestación de agentes basados en modelos de lenguaje para el desarrollo de software, capaz de transformar una especificación en lenguaje natural en un conjunto estructurado de tareas, ejecutar dichas tareas en entornos aislados e integrar sus resultados de manera verificable, trazable y supervisable. Como parte del análisis, se estudiará de forma exploratoria una política adaptativa para determinar la granularidad de la descomposición según las características de cada tarea.

Objetivos específicos

1. Definir una arquitectura de planificación basada en un grafo jerárquico e implementar un mecanismo de descomposición recursiva que produzca tareas, dependencias, criterios de aceptación e interfaces compartidas.
2. Implementar una ejecución aislada que asigne tareas a agentes, controle el alcance de sus modificaciones, ejecute validaciones y registre evidencia de los cambios realizados.
3. Implementar una integración incremental que recomponga los resultados según el grafo, conserve su trazabilidad y permita reparar conflictos a partir del objetivo y los contratos definidos durante la descomposición.
4. Proporcionar mecanismos de supervisión y recuperación para observar el progreso, registrar decisiones, intervenir ante fallas y reanudar ejecuciones.
5. Diseñar una política adaptativa de granularidad que considere la complejidad de cada tarea, información estructural del repositorio y el costo potencial de nuevas divisiones.
6. Analizar el funcionamiento de la plataforma mediante casos seleccionados de desarrollo de software y comparar descriptivamente los planes producidos por distintas configuraciones de granularidad.

2. Metodología

El trabajo combinará diseño de arquitectura, desarrollo incremental, verificación automatizada y análisis exploratorio. La implementación se organizará en módulos con responsabilidades explícitas y contratos entre ellos. El estado persistido y los eventos producidos por cada ejecución se utilizarán tanto para supervisar el producto como para reconstruir las decisiones adoptadas en los casos estudiados.

2.1. Descomposición

La entrada del sistema será una descripción de una funcionalidad sobre un repositorio de software. Un agente planificador analizará recursivamente cada nodo y decidirá si representa una unidad cohesiva y verificable o si requiere una nueva división. El resultado será un grafo dirigido acíclico (DAG) con dependencias, criterios de aceptación, alcance esperado e interfaces compartidas.

Las interfaces actuarán como contratos entre tareas: permitirán que agentes aislados trabajen sobre una frontera común sin necesitar los resultados concretos de sus pares. Las salidas del planificador se validarán de forma estructural y semántica antes de habilitar la ejecución.

2.2. Ejecución

Cada tarea hoja se ejecutará en un *worktree* de Git independiente. El agente recibirá su objetivo, los contratos que consume o produce, el contexto relevante del repositorio y las validaciones esperadas. Una vez finalizada la tarea, el sistema utilizará el estado real del repositorio para determinar los cambios realizados, comprobar que respeten el alcance autorizado y ejecutar pruebas o comandos de validación.

Las tareas sin dependencias incompatibles podrán ejecutarse en paralelo. La selección de cada grupo de ejecución considerará dependencias, superposición de archivos y señales de riesgo estructural. Ante una validación fallida, el agente podrá recibir el diagnóstico y realizar un intento acotado de reparación.

2.3. Integración y supervisión

Los resultados se integrarán de abajo hacia arriba siguiendo la jerarquía del grafo. Cada contribución conservará un commit identificable y será aplicada al nodo compuesto correspondiente. Cuando aparezca un conflicto, un agente de integración recibirá el objetivo del nodo padre, las interfaces compartidas, la intención de los hijos y los cambios involucrados. La reparación deberá respetar esos contratos y superar las validaciones del conjunto.

La plataforma persistirá el estado del proceso, sus eventos y las decisiones humanas. Una interfaz web permitirá revisar el plan, observar la evolución del grafo, responder preguntas y resolver situaciones que no puedan tratarse de manera autónoma.

2.4. Política adaptativa y análisis exploratorio

La granularidad, el modelo y el esfuerzo de razonamiento son decisiones de asignación de recursos. Las interfaces actuales permiten configurar los dos últimos [5, 6], y distintos trabajos estudian políticas para equilibrar costo y calidad [7, 8]. En Conductor, la granularidad es una decisión previa: define las tareas y condiciona su ejecución, paralelismo e integración.

Se implementará una política que combine una estimación determinista de complejidad con una rúbrica sensible al costo de dividir. Se considerarán el alcance, las interfaces, los archivos esperados y señales del repositorio, como símbolos e imports, frente al costo adicional de coordinación, integración y posibles conflictos. Cada recomendación conservará evidencia auditable.

El análisis exploratorio se organizará en dos líneas complementarias. La primera estudiará tres casos completos: una funcionalidad simple, otra de complejidad intermedia y una con solapamientos o conflictos esperables. Si los recursos lo permiten, se incorporará un cuarto caso centrado en la recuperación ante una falla. Para cada caso se describirán el DAG producido, las interfaces y dependencias, las waves de ejecución, las validaciones, los conflictos, las reparaciones, las intervenciones humanas y el resultado integrado. También se registrarán la duración y el consumo cuando puedan obtenerse de forma confiable.

La segunda línea analizará la granularidad sin ejecutar las tareas. Sobre un conjunto acotado de funcionalidades se compararán los planes generados por la política propuesta, la rúbrica adaptativa inicial y una granularidad fija intermedia. Se observarán la cantidad total de nodos, la profundidad y el ancho del DAG, las tareas atómicas y compuestas, las interfaces, las dependencias y los posibles solapamientos entre alcances.

Dentro de cada comparación se mantendrán constantes el repositorio, la funcionalidad y el modelo. Los resultados se interpretarán de manera descriptiva y cualitativa, sin pretensión de generalización estadística. La selección automática de modelos y esfuerzo quedará como trabajo futuro, y la comparación con un agente único quedará fuera del alcance.

3. Cronograma de actividades

Se adopta un cronograma tentativo de aproximadamente cuatro meses, iniciado durante la primera mitad de abril de 2026.

Actividad	Abr.	May.	Jun.	Jul.	Ago.
1. Relevamiento y definición de arquitectura	•	•			
2. Descomposición y contratos	•	•	•		
3. Ejecución aislada y validación		•	•	•	
4. Integración, persistencia y supervisión			•	•	
5. Política adaptativa de granularidad				•	•
6. Estudio de casos y análisis exploratorio				•	•
7. Documentación y redacción de la tesina		•	•	•	•

4. Resultados esperados

Se espera obtener una plataforma funcional capaz de transformar un objetivo de desarrollo en un plan estructurado, ejecutar sus tareas en entornos aislados e integrar los cambios producidos. El sistema deberá conservar evidencia de cada etapa y ofrecer puntos de supervisión y recuperación ante decisiones ambiguas o fallas.

El principal resultado técnico será la articulación de la descomposición, la ejecución y la integración en un producto completo. La tesina documentará la arquitectura, los contratos entre estos mecanismos, las decisiones de diseño adoptadas y las limitaciones identificadas durante su implementación.

El estudio de casos permitirá describir el comportamiento del sistema completo en escenarios de distinta complejidad, incluyendo su respuesta ante conflictos, reparaciones o decisiones humanas. El análisis de planificación documentará cómo varía la estructura del DAG entre configuraciones y aportará evidencia sobre la utilidad y los límites de la política adaptativa, sin formular generalizaciones estadísticas.

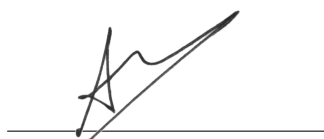
5. Vinculación con actividades de investigación o transferencia

El presente proyecto final no se encuentra vinculado formalmente a ninguna actividad de investigación, vinculación o extensión vigente en el Departamento de Ciencias e Ingeniería de la Computación (DCIC).

6. Firmas



Francisco Clarke
Alumno



Ana Gabriela Maguitman
Directora



Federico Martín Schmidt
Codirector

Referencias

- [1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [2] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174–15186, 2024.
- [3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [4] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, “An LLM compiler for parallel function calling,” in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, PMLR, 2024.
- [5] OpenAI, “Introducing GPT-5 for developers.” Documentación oficial, 2025. Accedido en julio de 2026.
- [6] Anthropic, “Adaptive thinking.” Documentación oficial de Claude API, 2026. Accedido en julio de 2026.
- [7] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” 2023.
- [8] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs from preference data,” in *International Conference on Learning Representations*, 2025.